
PART A: WHAT TO DO BEFORE DSA (Foundation Phase)

Week 1-2: JavaScript Absolute Essentials

Order	Topic	Mastery Check (do this before moving)
1	Variables (let, const, var) + scoping	Can explain difference between block/function scope
2	Data Types (primitive: string, number, boolean, null, undefined, symbol)	Know that arrays & objects are "objects"
3	Operators (arithmetic, comparison, logical, ternary)	Can write <code>a > b ? "yes" : "no"</code>
4	Conditionals (if/else, switch)	No issues
5	Loops (for, while, do-while, for...of, for...in)	Can iterate array and object separately
6	Functions (declaration, expression, arrow, parameters, return)	Can write a pure function
7	Arrays (create, access, push, pop, shift, unshift, slice, splice)	Can reverse an array manually
8	Objects (create, access, add/delete keys, nested objects)	Can loop over object properties
9	Array Methods (map, filter, reduce, forEach, find, some, every)	Critical – practice 10 problems only on these

Order	Topic	Mastery Check (do this before moving)
10	String Methods (split, join, slice, substring, toUpperCase, trim, includes)	Can reverse a string without built-in

Week 3: Problem-Solving Basics (Still before DSA)

Order	Topic	Practice Problems
1	Find max/min in array	5 problems
2	Remove duplicates from array	3 problems
3	Check palindrome (string + number)	4 problems
4	Count characters in string	3 problems
5	FizzBuzz	1 problem
6	Factorial (iterative + recursive)	2 problems
7	Fibonacci (iterative only first)	2 problems

Exit Criteria for Phase A: You can solve any of the above in under 10 minutes without looking up syntax.

PART B: DSA TOPICS IN EXACT ORDER (For Node.js Full-Stack)

Level 1: Foundational DSA (Must do before anything else)

Order	Topic	Why for Node.js	Time to spend
1	Time & Space Complexity (Big O)	To know if your code is slow	2 days

Order	Topic	Why for Node.js	Time to spend
2	Basic Recursion	File system traversal	3 days
3	Arrays (advanced) – sliding window, two pointers	API data processing	5 days
4	Strings (advanced) – substring problems, anagrams	Text validation, parsing	4 days
5	Hash Tables (Objects, Map, Set)	Caching, counting, lookups	4 days

After finishing Level 1 → You can solve Easy Leetcode problems

Level 2: Linear Data Structures

Order	Topic	Real Node.js use case
6	Stack	Undo/redo, route history, parsing
7	Queue	Task queues, request handling
8	Linked List (Singly + Doubly)	LRU cache, memory management

After Level 2 → You can build a simple task queue system

Level 3: Non-Linear Data Structures

Order	Topic	Real Node.js use case
9	Binary Trees (BST)	Database indexing (concept)

Order	Topic	Real Node.js use case
10	Tree Traversal (BFS, DFS)	Nested comments, folder structure
11	Heaps / Priority Queue	Job scheduling, task prioritization

After Level 3 → You can build a file tree viewer

Level 4: Algorithms (Optional but good)

Order	Topic	Priority for full-stack
12	Binary Search	High – search in sorted data
13	Sorting (Quick, Merge, Bubble – understand, rarely implement)	Medium
14	Recursive Backtracking	Medium – path finding, permutations
15	Graphs (BFS, DFS)	Low – social features
16	Dynamic Programming	Very Low (rarely asked for full-stack)

PART C: WHAT TO DO AFTER EACH TOPIC (Practical Integration)

After Arrays + Hash Tables → Build:

- API endpoint that counts word frequency in text

After Stack → Build:

- Browser back/forward button simulation

After Queue → Build:

- In-memory task queue with priority

After Trees + Recursion → Build:

- Folder structure printer (like tree command)

After Sorting + Search → Build:

- Search API with filters and sorting

PART D: BEGINNER RULES (Strictly Follow)

Phase	Rule
Before coding	Write 3 example inputs + outputs on paper
During coding	Add console.log after every 2-3 lines
After solving	Can you do it in less time? Less memory?
Stuck >20 min	Look at solution, understand, close it, rewrite
Day after	Re-solve same problem without looking
Week after	Solve 2 similar problems on LeetCode

PART E: TOPIC-WISE CHECKLIST (Print this)

Before DSA (when done)

- Variables & scoping
- Loops + conditionals
- Functions (all types)
- Array methods (map/filter/reduce)
- String methods
- Objects + nested objects
- 10 basic array/string problems

DSA Level 1

- Big O ($O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$)
- Recursion base + recursive case
- Two pointers in array
- Sliding window
- Hash table (Map vs Object)
- Set for uniqueness

DSA Level 2

- Stack (array as stack)
- Queue (array as queue + optimization)
- Linked List (Node class + traversal)

DSA Level 3

- Binary tree node
- BFS traversal
- DFS traversal (pre/in/post order)
- Min heap / priority queue

DSA Level 4 (Optional)

- Binary search
- Quick sort (understand only)
- Graph BFS/DFS basics

PART F: SAMPLE WEEK 1 SCHEDULE (Topic-wise)

Day	Morning (2hr)	Evening (1hr)
Mon	Variables + Data Types	5 small problems
Tue	Loops + Conditionals	FizzBuzz + patterns
Wed	Functions (all types)	5 function problems

Day	Morning (2hr)	Evening (1hr)
Thu	Array methods (map/filter)	10 array exercises
Fri	Reduce (most important)	5 reduce problems
Sat	String methods	5 string problems
Sun	10 mixed problems (arrays + strings + objects)	Review weak areas

Week 2: Start DSA Level 1 (Big O + Recursion)

PART G: DSA-LINKED TOPICS FOR NODE.JS (Learn AFTER DSA)

These are not pure DSA but connect directly:

After learning this DSA topic	Then learn this Node.js concept
Queues	BullMQ / RabbitMQ
Hash Tables	Redis caching
Recursion + Trees	File system (fs.readdir recursive)
Event Loop basics	Async DSA patterns (callback hell → promises)
Binary Search	Database query optimization
Stacks	Middleware execution (Express.js)

Do not touch LeetCode until you finish **Part A (Week 1-3)** – or you'll get frustrated.